MateriaLens Architecture

Proposed MVP architecture for source-grounded research over a defined Materia Medica corpus

1. Purpose and scope

This document describes the proposed architecture for MateriaLens, an AI research assistant intended to help homeopathic practitioners find, compare and verify passages in a defined corpus of Materia Medica and repertory/reference texts. It is a design specification, not evidence that a production system has been implemented or validated.

2. High-level architecture

Stage	MVP responsibility	Key output
1. Source corpus	A small set of authorized or public-domain texts; record title, author, edition/year and reuse status.	Approved source inventory
2. Ingestion and OCR	Import PDF/text; OCR scanned pages where needed; clean repeated headers/footers and preserve page references.	Normalized text with source metadata
3. Structure-aware chunking	Split by chapter/section and local context; initial working assumption: ~400 tokens per chunk with ~80-token overlap.	Passages with stable IDs and location metadata
4. Indexing	Create embeddings and a lexical keyword index; retain document, edition, chapter and page metadata.	Searchable hybrid index
5. Query processing	Accept natural-language research questions; ask a clarifying question if the query is too ambiguous.	Normalized query
6. Hybrid retrieval	Combine semantic and keyword retrieval; use reciprocal-rank fusion as an initial ranking approach.	Ranked candidate passages
7. Evidence selection	Select a small set of relevant passages (initial Top-5 assumption); preserve source locations and conflicts.	Evidence context for generation
8. Answer generation	Use Gemini 2.5 Flash as the proposed MVP LLM; answer only from supplied retrieved evidence.	Concise synthesis and citations
9. User interface	Show synthesis, Evidence Matrix, source snippets, metadata, conflict note and feedback controls.	Verifiable research response
10. Observability	Log query IDs, chunk IDs, retrieval/ranking signals, citation checks, latency, token usage and feedback.	Debugging and evaluation data

3. Query flow

1. The practitioner enters a symptom, concept or research question.
2. The system checks whether the query is specific enough to search; ambiguous requests trigger a clarifying question.
3. Hybrid retrieval searches the semantic and keyword indexes and combines candidate rankings.
4. The system selects relevant passages while preserving source title, edition and page/section metadata.
5. The LLM produces a concise synthesis using only the retrieved passages and links each factual claim to its supporting source.
6. The interface displays the answer, Evidence Matrix, source passages and any disagreement or evidence gaps.
7. Automated checks and user feedback are captured for evaluation; insufficient evidence triggers abstention rather than guessing.

4. Proposed technology choices

Component	Proposed choice / approach	Rationale and caveat
LLM	Gemini 2.5 Flash	Proposed cost-conscious model for concise synthesis. Validate citation fidelity and answer quality in the evaluation set before committing.
Embeddings	A lightweight embedding model selected after a small retrieval benchmark	The final model is not treated as validated until compared on representative corpus queries.
Vector store	Chroma for an initial prototype, paired with a lexical index	Simple prototype path; re-evaluate if scale, access control or operational needs change.
Retrieval	Semantic + keyword retrieval with reciprocal-rank fusion	Supports both concept similarity and exact terms/abbreviations; benchmark against simpler baselines.
Chunking	~400 tokens, ~80-token overlap; split at structural boundaries where possible	Starting assumption only. Test whether remedy abbreviations, rubric context and page references survive chunking.
Reranking	Optional lightweight reranking after baseline evaluation	Do not add complexity unless it improves evidence retrieval on the labeled query set.

5. Data and source controls

- Use only texts whose access and reuse permissions have been checked. Public availability does not automatically mean unrestricted redistribution.
- Store source title, author, edition/year, chapter/section, page/location, version/hash and ingestion date with each passage.
- Retain OCR confidence or quality flags for scanned material; sample-check OCR before indexing.
- Re-index changed documents and preserve version information so a citation can be traced to the source edition used.
- Do not place full copyrighted source texts in the public repository unless redistribution is permitted.

6. Safety and failure handling

- No autonomous diagnosis, prescribing or treatment recommendations.
- If retrieved evidence is insufficient, contradictory or not traceable, state the limitation and do not manufacture a confident answer.
- For requests requiring clinical judgement, direct the user to make the decision independently as a qualified practitioner; the tool does not make or approve clinical decisions.
- Do not retain case details beyond the active session unless the user explicitly saves them. Define deletion behaviour before any real user data is collected.

7. Observability and evaluation hooks

- Capture query ID, source/chunk IDs, retrieval ranks, reranker scores (if used), citation IDs, latency, token usage and feedback labels.
- Measure Precision@5 and Recall@5 on labeled queries; review citation support and groundedness with human audits.
- Track insufficient-evidence and ambiguity cases separately from answerable queries.
- Use a repeatable evaluation set before each meaningful change to chunking, retrieval, prompts or model.

8. Open decisions before implementation

- Confirm the exact corpus, editions, file formats and reuse permissions.
- Benchmark candidate embedding models and decide whether reranking adds value.
- Define evidence-confidence thresholds using evaluation results rather than arbitrary production claims.
- Confirm the deployment, authentication, retention and account-deletion design before handling real case data.